1) Different Test Metrics :

Test Metrics:

* Test metrics are numbers (measurements) used to check quality of software, progress of testing and performance of testing process.

Test metrics ⇒ data that helps us understand how testing is going.

Key aspects of test metrics:

Quantifiable Data:

Metrics convert testing activities into numerical values.

Example:

*Instead of saying "many bugs", we say 10 defects.

* Helps in easy comparision, Tracking improvements.

Purpose:

Metrics are created to:

* Check effectiveness of testing.
* Track testing progress.
* Measure product Quality
* Support decision-making.

So, manager don't guess- they use data to decide.

Scope:

* Metrics are used in all phases of SDLC
* From development → testing → release.

Ensures product is checked at every stage.

Reporting:

* Metrics help in creating reports.
* These reports are shared with
  - ↳ Testers
  - ↳ Managers
  - ↳ Stakeholders.
* Gives clear idea of what is completed, what is pending, what issues exist.

Common Types of Test Metrics:

Defect metrics:

Used to measure defects (bugs)

Defect density:

* Number of defects per unit of code.
* Helps understand how many bugs exist in a specific part.

Defect Leakage:

* Defects missed during testing but found by users.
* shows testing quality.

If leakage is high → testing is weak.

Test Coverage Metrics: Test coverage metrics measure how much of the software's code or functionality has been tested.

Test Coverage: The percentage of total code that is executed by a test suite.

Test Becution Coverage: The percentage of planned test cases that have been executed.

Test Effort metrics: Test effort metrics focus on the resources, time and cost associated with testing.

Test Execution Time: The total time taken to execute test cases.

Cost per test case / Defect: Cost involved in testing or fixing bugs. It helps in Budget planning, Resource management.

Importance of Metrics in Software Testing:

Early problem Identification:

By measuring metrics such as defects density and defect arrival rate, testing terms can spot trends and pattern easily.

Resource Allocation:

Metrics identify regions where testing efforts are most needed. ensuring resources are concentrated and important areas.

Monitoring progress:

Metrics are useful instruments for monitoring the advancement of testing.. insight into number of test cases run and completion rate.

Continuous improvement:

Metrics offer input on the testing procedure, which helps to foster a culture of continuous development.

Types of software Testing metrics.

Software Testing metrics are divided into three categories.

process Metrics:

A project's characteristics and execution are defined by process metrics.. important for SDLC improvement:

product metrics:

A product's size, design, performance, quality, and complexity are defined by product metrics.

project metrics:

Used to assess a project's overall quality. estimate resources, cost, productivity and flaws.

Manual Test Metrics

* Base Metrics:

Raw data collected during testing

Examples:

Total test cases

Executed test cases.

* Calculated Metrics:

Derived from base metrics.

Converted into meaningful data.

Example:

% executed

% passed.

It helps to track tester performance and module progress.

Formulas for Test Metrics:

Test cases Executed: Shows how much testing is completed.

Test case Effectiveness: Shows how effective test cases are in finding defects.

passed / Failed / Blocked %. : Shows distribution of test results. (6)

Fixed Defects : Shows how many defects are resolved

All formulas help in performance measurement.

2) Test Estimation.

↳ Test estimation is the process of predicting

* Time
* Effort
* Resources.

↳ It helps in

* planning schedule
* Budgeting
* Resource allocation.

Test Estimation includes :

* Effort :
  ↳ Refers to the total work required.
  ↳ Measured in
    * Hours
    * story points.
  ↳ It tells how much work testers need to do.

* Time :
  ↳ Refers to the duration to complete testing.

* Measured in
  - ↳ Days
  - ↳ Calender Time
* Helps in meeting deadlines.

Cost :

* Refers to the budget required.
* Includes :
  - ↳ Resources
  - ↳ Tools
  - ↳ Infrastructure.
* Helps in financial planning.

Key Steps for Effective Estimation:

Work Breakdown:

* Divide testing into small, managable tasks
* Makes estimation easier and more accurate.

Task Identification:

* Identify activities such as
  - ↳ Test planning
  - ↳ Test analysis.
  - ↳ Test design
  - ↳ Test Execution.
* Ensures all work is considered.

Estimation:

↳ Assign effort, Time, cost to each task.

↳ Then combine to get total estimation.

Factor Analysis:

↳ Consider factors like:

* Project complexity.
* Development model.
* Team skills and experience.

Buffer Inclusion.

↳ Add extra time for

* Unexpected issues.
* Changes.

prevents delay in projects.

Continuous Review:

↳ Compare estimated values and actual results. Helps improve future estimation.

Communication:

↳ Clearly explain assumptions, basis of estimation and Trade-offs.

↳ It is important for stakeholders understanding.

Test Estimation Best Practices:

Resource planning: Ensure proper allocation of team members.

past project Reference: Use previous experience to improve accuracy.

Buffer Time: Add extra time for unexpected situations.

stick to estimation: Avoid frequent changes & maintain stability.

Bug cycle consideration: Bug fixing takes time and must be included in estimation.

Scope of project: Understand small vs large projects. Large projects needs more test cases data and documentation.

Load Testing plan: Include time for performance Testing.

parallel Testing: Testing multiple versions reduce efforts. It helps in better estimation.